

1. What did you ask the AI to do, and what did you write or decide yourself? I treated the AI a lot like a junior developer on my team. I asked it to handle the repetitive heavy lifting—writing boilerplate code, setting up the basic folder structures, and typing out standard algorithms.

However, I was the one steering the ship. I made all the core architectural decisions, like choosing the tech stack and breaking the project down into four distinct microservices. I also set the guardrails: instead of letting the AI write code freely, I created a strict, step-by-step checklist for it to follow. I reviewed its work at every major milestone and personally tested the infrastructure before allowing it to move forward. Essentially, I did the system design and planning, and the AI acted as my hands on the keyboard.

2. Where did you override, correct, or throw away the AI's output — and why? I had to step in whenever the AI tried to take the easy way out or got stuck on environment quirks.

For example, when we were building the feature that lets users pick a custom alias, the AI initially wanted to just silently return the existing link if an alias was already taken. I overrode that and forced it to return a proper "Conflict" error. I wanted to make sure users couldn't accidentally overwrite or assume ownership of someone else's short link.

Another time, the AI got completely stuck because our analytics database refused to connect. It couldn't figure it out, so I had to step in, dig into my local network logs, and realize Zscaler was blocking the default port. I instructed the AI to throw away its configuration, remap the ports to bypass the VPN, and the issue was fixed immediately.

3. The two or three biggest trade-offs you made, and the alternatives you considered.

- **Tracking analytics:** The simplest way to track link clicks would have been to just update a counter in our main database every time someone clicked a link. I decided against this because if a link went viral, all those simultaneous updates would overwhelm the database and slow down the app. Instead, I set up a background streaming queue to collect the clicks, and a separate

data warehouse to analyze them. It took more effort to build, but it guarantees the app stays lightning-fast even during massive traffic spikes.

- **Generating unique IDs:** A common approach is to let the database automatically generate a sequential ID (1, 2, 3...) for every new link. But in a large-scale system, having a single database do this creates a massive bottleneck. Instead, I chose to build a standalone "Key Generation Service" that generates unique, randomized IDs mathematically across multiple servers. It adds a bit of complexity to the system, but it makes our ID generation completely decentralized and highly scalable.

4. What's missing, or what you'd do with another day? If I had another day, I'd focus on the user experience and deployment. Right now, the backend is rock-solid and fully functional via the command line, but I'd love to build a clean, modern web interface so people can actually click around, create links, and view their analytics charts visually.

I'd also spend time wrapping our application code into Docker containers so that the entire project can be deployed to the cloud with a single click, and I'd set up an automated testing pipeline in GitHub so we can automatically catch bugs before they ever reach production.